

Introduction to adsalmd

Contents

What this package does	1
A toy system	2
The whole pipeline in one call	2
Opening up the pipeline	3
Diagnostics	6
Choosing thresholds	6
Parallel runs	7
Where to go next	7

```
library(adsalmd)
```

What this package does

`adsalmd` delineates **local labour market areas (LMAs)** — often called “markets” in this vignette — from two inputs:

- an **origin-destination matrix** W , where $W[i, j]$ is the number of people who live in basic geographic unit (BGU) i and work in BGU j ;
- a spatial **adjacency matrix** `adj`, recording which BGUs share a border.

The output is a partition of BGUs into markets that are internally cohesive (most commuting stays inside the market), self-contained (the market supplies its own workers and its own jobs), population-sized within bounds you choose, and spatially contiguous.

The method runs in three stages, and the package exposes both the whole pipeline and every individual stage:

1. **SARA** (Spatially Adaptive Random Aggregation) builds a feasible starting partition by repeatedly merging the least valid market into a strong neighbour.
2. **AdSA-ALMD** improves it with simulated annealing, using eleven group-based perturbation operators and an adaptive cooling schedule.
3. A deterministic **refinement** pass reassigns any BGU that would clearly fit better in a neighbouring market.

`run_adsa_pipeline()` chains all three. This vignette walks through the whole workflow once end to end, then opens up each stage so you can see what it is actually doing. For the full argument list of every function shown here, see its help page (e.g. `?run_adsa_pipeline`) — this vignette is about how the pieces fit together, not an exhaustive reference.

A toy system

Six BGUs in a line, 1 - 2 - 3 - 4 - 5 - 6. Commuting is heavy within {1, 2, 3} and within {4, 5, 6}, and almost absent between the two halves, so the “right answer” is the obvious two-market split — which makes it easy to check that every function below is doing what you’d expect.

```
adj <- matrix(0L, 6, 6)
adj[cbind(1:5, 2:6)] <- 1L
adj <- adj + t(adj)      # symmetric: BGU i adjacent to i + 1

W <- matrix(1, 6, 6)    # a little commuting everywhere
W[1:3, 1:3] <- 30       # heavy commuting inside {1,2,3}
W[4:6, 4:6] <- 30       # heavy commuting inside {4,5,6}
diag(W) <- 60           # most people work where they live

row_W <- rowSums(W)      # residents of each BGU (precomputed once)
col_W <- colSums(W)      # jobs in each BGU (precomputed once)

adj
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,]    0    1    0    0    0    0
#> [2,]    1    0    1    0    0    0
#> [3,]    0    1    0    1    0    0
#> [4,]    0    0    1    0    1    0
#> [5,]    0    0    0    1    0    1
#> [6,]    0    0    0    0    1    0
W
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,]   60   30   30    1    1    1
#> [2,]   30   60   30    1    1    1
#> [3,]   30   30   60    1    1    1
#> [4,]    1    1    1   60   30   30
#> [5,]    1    1    1   30   60   30
#> [6,]    1    1    1   30   30   60
```

row_W and col_W are computed once here and threaded through almost every function in the package. Passing stale values after modifying W silently produces wrong metrics, so recompute them together.

The “true” partition for this toy system is:

```
true_sol <- c(1, 1, 1, 2, 2, 2)
```

The whole pipeline in one call

For most uses, `run_adsa_pipeline()` is the only entry point you need. It requires four thresholds: `SC_min` (minimum self-containment), `Pop_min` and `Pop_max` (population floor and ceiling), and `r` (the penalty steepness exponent, 3 or 4 in the paper).

```
set.seed(2026)
res <- run_adsa_pipeline(
  W = W, adj = adj, row_W = row_W, col_W = col_W,
  SC_min = 0.55,
  Pop_min = 200,
```

```

Pop_max = 500,
r = 3,
L = 5L, l = 10L, T0_samples = 20L,    # kept small for this vignette
verbose = FALSE
)
#> Trial      1 | Best F =  1.4277 | T = 0.028637 | m = 2
#> Trial      2 | Best F =  1.4277 | T = 0.019522 | m = 2
#> Trial      3 | Best F =  1.4277 | T = 0.009904 | m = 2
#> Trial      4 | Best F =  1.4277 | T = 0.004410 | m = 2
#> Trial      5 | Best F =  1.4277 | T = 0.002048 | m = 2

```

```

res$best_sol
#> [1] 1 1 1 2 2 2
res$lma_metrics
#>   cluster pop    scss    scds    sc n_bgus
#> 1      1 369 0.9756 0.9756 0.9756      3
#> 2      2 369 0.9756 0.9756 0.9756      3

```

Did it recover the two-market answer? Partition labels are arbitrary, so compare with `partitions_match()` rather than `identical()`:

```

partitions_match(true_sol, res$best_sol)$match
#> [1] TRUE

```

`L`, `l` and `T0_samples` control the search effort and are set very small above purely so this vignette builds quickly. On real data, start with the defaults (`L = 1000`, `l = 20`, `T0_samples = 100`) or `AdSA_params`, and use a handful of independent runs via `run_adsa_pipeline_parallel()` rather than one long run — see the Parallel runs section below.

Opening up the pipeline

Metrics

Every market is scored on **cohesion** and **self-containment**.

```

## Cohesion index of BGU 1 within its own market, vs. against the far market
compute_ci(1, c(2, 3), W, row_W, col_W)
#> [1] 0.238
compute_ci(1, c(4, 5, 6), W, row_W, col_W)
#> [1] 0.0003966

```

`compute_sc()` gives supply-side (`scss`), demand-side (`scds`), and their minimum (`sc`) for a market:

```

compute_sc(1:3, W, row_W, col_W)
#> $scss
#> [1] 0.9756
#>
#> $scds
#> [1] 0.9756
#>
#> $sc
#> [1] 0.9756

```

`build_lma_df()` puts population and self-containment for every market in a partition into one table — the object to inspect first whenever a result looks wrong:

```
build_lma_df(true_sol, W, row_W, col_W)
#>   cluster pop  scss  scds   sc n_bgus
#> 1      1 369 0.9756 0.9756 0.9756    3
#> 2      2 369 0.9756 0.9756 0.9756    3
```

The objective function

The search maximises `compute_objective()`: global cohesion minus penalties, zeroed outright if any market is non-contiguous or violates a population bound.

```
compute_objective(true_sol, W, row_W, col_W,
                  SC_min = 0.55, Pop_min = 100, r = 3, adj = adj)
#> [1] 1.428

## A non-contiguous market scores exactly 0
compute_objective(c(1, 2, 1, 2, 2, 2), W, row_W, col_W,
                  SC_min = 0.55, Pop_min = 100, r = 3, adj = adj)
#> [1] 0
```

Stage 1: SARA

`sara()` merges markets until every one clears the validity criterion (self-containment, population floor, contiguity). `sara_iterated()` wraps it and additionally breaks up any market over `Pop_max`:

```
set.seed(42)
init_sol <- sara_iterated(
  W, adj, row_W, col_W,
  SC_min = 0.55, Pop_min = 200, Pop_max = 500,
  max_it = 5, verbose = FALSE
)
init_sol
#> [1] 1 1 2 2 2 2
build_lma_df(init_sol, W, row_W, col_W)
#>   cluster pop  scss  scds   sc n_bgus
#> 1      1 246 0.7317 0.7317 0.7317    2
#> 2      2 492 0.8659 0.8659 0.8659    4
```

Stage 2: AdSA-ALMD annealing

`adsa_almd()` takes that starting partition and improves it. It perturbs the partition with eleven operators (`operator_1()` ... `operator_11()`) — merging weak markets, growing small ones, splitting oversized ones, swapping border BGUs — dispatched by `apply_random_operator()`. Several of the operators read a `params` list from the global environment rather than taking thresholds as arguments, so calling this stage directly (as opposed to through `run_adsa_pipeline()`, which sets it up for you) needs one in scope:

```
params <- list(SC_min = 0.55, Pop_min = 200, Pop_max = 500)
```

```
set.seed(43)
ann <- suppressMessages(adsa_almd(
  init_sol = init_sol, W = W, adj = adj, row_W = row_W, col_W = col_W,
  SC_min = 0.55, Pop_min = 200, Pop_max = 500,
  r = 3, L = 5L, l = 10L, TO_samples = 20L, verbose = FALSE
))
```

```
ann$best_sol
#> [1] 1 1 1 2 2 2
ann$n_clusters
#> [1] 2
```

Which operators actually contributed?

```
print_op_tracker(ann$op_tracker)
#>
#> === Operator Performance Report ===
#> Operator      Called Unchanged  Changed Accepted Improved  Acc%
#> -----
#> operator_1      5         4         1         0         0         0
#> operator_2      3         0         3         0         0         0
#> operator_3      1         0         1         0         0         0
#> operator_4      5         0         5         0         0         0
#> operator_5      5         0         5         0         0         0
#> operator_6      8         6         2         0         0         0
#> operator_7      6         0         6         0         0         0
#> operator_8      4         4         0         0         0         0
#> operator_9      3         3         0         0         0         0
#> operator_10     5         5         0         0         0         0
#> operator_11     5         5         0         0         0         0
#> -----
#> TOTAL          50        27        23         0         0         0
#> =====
```

The run also keeps an **elite pool** of near-best solutions, useful when the objective alone doesn't clearly separate a few candidate partitions:

```
ann$elite$fits
#> [1] 1.428 1.428 1.428 1.428 1.428
```

Stage 3: refinement

A final deterministic pass reassigns any BGU that would clearly fit better in a neighbouring market:

```
refined <- refine_misallocated_bgus(ann$best_sol, W, adj, row_W, col_W)
refined
#> [1] 1 1 1 2 2 2
```

`run_adsa_pipeline()` does exactly this — SARA, then annealing, then refinement — and recomputes `gci`, `base_fitness`, `n_clusters` and `lma_metrics` for the refined partition, which is why its result matches `res` above.

Diagnostics

`count_misallocated()` counts BGUs that would fit better elsewhere without a substantial loss of self-containment for the receiving market — the comparable-across-methods quality measure used in the paper (Table 3):

```
count_misallocated(res$best_sol, W, adj, row_W, col_W)
#> [1] 0
```

`build_results_table()` compares several methods' results side by side, using this package's own metrics for all of them so the numbers are on a common scale:

```
as_result <- function(sol) list(
  best_sol      = sol,
  base_fitness  = compute_base_fitness(sol, W, row_W, col_W),
  gci           = compute_gci(sol, W, row_W, col_W),
  n_clusters    = n_clusters(sol),
  lma_metrics   = build_lma_df(sol, W, row_W, col_W)
)

build_results_table(
  list(
    `AdSA-ALMD` = as_result(res$best_sol),
    Naive       = as_result(c(1, 1, 1, 1, 2, 2)),
    Singletons  = as_result(1:6)
  ),
  W, adj, row_W, col_W
)

#>      Method Base_Fitness   GCI  SCSS  SCDS m Misallocated_BGU
#> 1 AdSA-ALMD      2.855 1.428 0.976 0.976 2              0
#> 2 Naive          1.461 0.730 0.799 0.799 2              1
#> 3 Singletons     0.000 0.000 0.488 0.488 6              6
```

Choosing thresholds

`SC_min`, `Pop_min`, `Pop_max` and `Pop_tar` were supplied by hand above. If you already trust some partition of your data — a synthetic test case, a previous method's output, a business rule — `suggest_thresholds()` reads plausible values off it instead of guessing:

```
th <- suggest_thresholds(true_sol, W, row_W, col_W)
th[c("SC_min", "Pop_min", "Pop_max", "Pop_tar")]
#> $SC_min
#> [1] 0.878
#>
#> $Pop_min
#> [1] 332.1
#>
#> $Pop_max
#> [1] 405.9
#>
#> $Pop_tar
#> [1] 369
```

```
th$lma_df
#>   cluster pop    scss    scds    sc n_bgus
#> 1      1 369 0.9756 0.9756 0.9756      3
#> 2      2 369 0.9756 0.9756 0.9756      3
```

Treat this as a starting point, not a substitute for the grid search described in the paper — every bound is set by whichever single market in your reference partition is most extreme on that measure, so look at `lma_df` before trusting the numbers.

Parallel runs

Because the search is stochastic, a single run is not the recommended way to produce a final delineation. `run_adsa_pipeline_parallel()` runs the whole pipeline once per seed on a PSOCK cluster and keeps the best:

```
out <- run_adsa_pipeline_parallel(
  n_runs = 8,
  W = W, adj = adj, row_W = row_W, col_W = col_W,
  SC_min = 0.55, Pop_min = 200, Pop_max = 500,
  r = 3, L = 200L, l = 20L,
  n_cores = 4
)

out$summary      # best / mean / sd of base fitness across runs
out$best_run$best_sol
```

This needs the package installed (`install.packages(".", repos = NULL, type = "source")`), since each worker loads it with `library()`, so it is not run in this vignette. A small `sd_fitness` in `out$summary` means the search keeps finding the same structure; a large one means your map has several competing partitions and the single best run may be luck — compare the individual runs' `best_sol` with `partitions_match()` before trusting one.

Where to go next

- `?adsalmd` — the package-level help page, with the full notation table.
- `?operator_1` through `?operator_11` — what each perturbation operator does and when it fires.
- `?AdSA_params` — the default thresholds used for the Queensland SA2 study this method was built for.
- `?partitions_match` — comparing a result against a known answer in detail, including how to read a mismatch.